

Problem 2.1

Solution:

(a) Pseudocode:

```

C = [[0] * (n+1)] * 3
for j in range(1, n+1):      # iterate j in increasing order
    roll C
    for i in range(n, 0, -1): # iterate i in decreasing order
        term1 = C[1][i-1] if (i-1>0 and j-1>0) else 0
        term2 = C[0][i+2] if (i+2<=n and j-2>0) else 0
        term3 = C[2][i+j] if i+j<=n else 0
        C[2][i] = max(term1, term2, term3)
return C[2][n]

```

Time complexity is $O(n^2)$.Space complexity is $O(n)$ using rolling array.

(b) Pseudocode:

```

C = [[0] * n for _ in range(n)]
root = n / 2
for j in range(n, 0, -1):
    # evaluate from right to left
    for i in range(min(n-j, root), 0, -1):
        # evaluate from two sides to center
        C[root-i][j+i] = evaluate_min(C, root-i, j)
        C[root+i][j+i] = evaluate_min(C, root+i, j)
    C[root][j] = evaluate_min(C, root, j)
return C[root][1]

```

Iteration takes $O(n^2)$ steps, each step takes $O(n)$ time to evaluate minimum value, total time complexity is $O(n^3)$.Space complexity is $O(n^2)$.